

Escola Naval — Curso EN-AEL

Algoritmos e Estruturas de Dados 2025–26

Grafo de Conhecimento para Cyber Threat Intelligence

Relatório Técnico — Nível 3: Algoritmos de Análise

Daniel Zayika

1 de junho de 2026

Conteúdo

1 Introdução

Este relatório documenta a implementação e os resultados do **Nível 3** do projeto, que acrescenta algoritmos de análise de grafos ao sistema de Knowledge Graph para Cyber Threat Intelligence (CTI) desenvolvido nos níveis anteriores.

O sistema representa entidades de segurança (atores, malware, indicadores, campanhas, vulnerabilidades) como nós de um grafo dirigido, com relações tipadas entre elas. O Nível 3 transforma este repositório de dados numa *ferramenta analítica*: é agora possível atravessar o grafo, descobrir caminhos entre entidades, quantificar a relevância de cada nó e resolver aliases de atores reportados sob nomes diferentes.

O dataset sintético utilizado contém **22 nós** e **26 relações**, modelando dois cenários reais: o ataque *SolarWinds Supply Chain* (2020), atribuído ao APT29/Cozy Bear, e a campanha *WannaCry* (2017), atribuída ao Lazarus Group.

2 Estrutura do Código

O código está organizado nos seguintes módulos:

Ficheiro	Responsabilidade
src/types.h	Enumerações <code>EntityType</code> e <code>RelationType</code> ; matriz de relações válidas
src/node.h	Estrutura <code>Node</code> (id, label, tipo, propriedades)
src/edge.h	Estrutura <code>Edge</code> (origem, destino, relação, descrição)
src/graph.h/.cpp	Classe <code>Graph</code> : lista de adjacência, inserção, consultas
src/database.h/.cpp	Camada SQLite: persistência e carregamento do grafo
src/algorithms.h/.cpp	Algoritmos: BFS, DFS, caminho mais curto, Page-Rank, resolução de entidades, consultas analíticas
src/synthetic_data.h/.cpp	Dataset sintético com entidades e relações CTI reais
src/tests.h/.cpp	Suite de testes automáticos
main.cpp	Ponto de entrada; demonstração em consola

3 Algoritmos Implementados

3.1 Travessia em Largura (BFS)

A BFS (*Breadth-First Search*) explora o grafo nível a nível a partir de um nó inicial, utilizando uma fila (`std::queue`). Visita primeiro todos os vizinhos diretos, depois os vizinhos dos vizinhos, e assim sucessivamente.

Complexidade: $O(V + E)$, onde V é o número de nós e E o número de arestas.

Resultado obtido (partindo de apt29, grafo dirigido):

```
apt29 -> camp-solarwinds -> id-nato -> id-usgov -> mimikatz -> sunburst
      -> atk-cred-dump -> atk-supply-chain -> teardrop
```

A BFS garante que os nós mais próximos de **apt29** são visitados primeiro: a campanha e os alvos diretos aparecem antes do malware e das técnicas de ataque, que se encontram a maior distância no grafo.

3.2 Travessia em Profundidade (DFS)

A DFS (*Depth-First Search*) explora cada ramo do grafo até ao fim antes de recuar, utilizando uma pilha (`std::stack`).

Complexidade: $O(V + E)$.

Resultado obtido (partindo de **apt29**, grafo dirigido):

```
apt29 -> camp-solarwinds -> id-usgov -> sunburst -> atk-supply-chain
      -> teardrop -> id-nato -> mimikatz -> atk-cred-dump
```

Ao contrário da BFS, a DFS segue imediatamente a cadeia **camp-solarwinds** → **sunburst** → **atk-supply-chain** → **teardrop** em profundidade, revelando a cadeia de ataque completa antes de voltar para os alvos.

3.3 Caminho Mais Curto

Implementado com BFS sobre o grafo não ponderado (cada aresta tem custo unitário). O algoritmo regista o nó pai de cada nó visitado e reconstrói o caminho por retrocesso (*backtracking*).

Complexidade: $O(V + E)$.

Resultados obtidos:

Origem	Destino	Caminho
ioc-hash-sunburst	apt29	ioc-hash-sunburst → sunburst → apt29
ioc-ip-91	id-nato	ioc-ip-91 → apt28 → id-nato
ioc-hash-wannacry	vuln-ms17-010	ioc-hash-wannacry → wannacry → vuln-ms17-010

Os três caminhos têm comprimento 2 (três nós), o que reflete a estrutura do dataset: os indicadores ligam-se diretamente ao malware/ator, que por sua vez se liga ao alvo ou vulnerabilidade. Estes caminhos curtos são exatamente o que um analista CTI precisa para justificar uma atribuição.

3.4 PageRank

O PageRank quantifica a importância de cada nó com base na estrutura de ligações do grafo. Um nó é relevante se muitos nós relevantes apontam para ele. A fórmula iterativa é:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in \text{In}(u)} \frac{PR(v)}{\text{Out}(v)}$$

onde $d = 0,85$ é o fator de amortecimento, N o número total de nós, $\text{In}(u)$ o conjunto de nós que apontam para u e $\text{Out}(v)$ o grau de saída de v . O algoritmo inclui

tratamento de *dangling nodes* (nós sem arestas de saída), redistribuindo a sua massa uniformemente.

Complexidade por iteração: $O(V + E)$. Converge tipicamente em menos de 100 iterações.

Top-8 entidades por PageRank:

#	Entidade	Score	Tipo
1	sunburst	0.114667	malware
2	camp-solarwinds	0.096785	campaign
3	atk-supply-chain	0.070113	attack-pattern
4	teardrop	0.070113	malware
5	vuln-ms17-010	0.070058	vulnerability
6	id-usgov	0.069236	identity
7	wannacry	0.057269	malware
8	atk-cred-dump	0.045267	attack-pattern

Interpretação: O SUNBURST lidera com grande margem porque é referenciado por múltiplas entidades de alto peso: apt29, dukes, camp-solarwinds e dois indicadores IoC. A campanha *SolarWinds* ocupa o segundo lugar por ser o hub central do maior cluster do grafo. A vuln-ms17-010 (EternalBlue) surge no top-5 apesar de pertencer ao cluster WannaCry, que é menor, porque é um ponto de convergência único nesse cluster — apenas o WannaCry aponta para ela, mas o WannaCry tem alto score por si próprio.

3.5 Resolução de Entidades

O sistema implementa resolução de aliases com dois mecanismos:

1. **resolveByName:** normaliza a query (minúsculas, apenas alfanuméricos) e compara com todas as variantes de cada nó: id, label, partes do label separadas por “/”, e a propriedade `alias`.
2. **findPotentialDuplicates:** agrupa nós que partilham pelo menos uma variante normalizada em comum.

Resultados obtidos:

```
resolveByName("APT29")      -> [apt29] "APT29 / Cozy Bear"
resolveByName("Cozy Bear") -> [apt29] "APT29 / Cozy Bear"
                           [dukes] "The Dukes"
```

Duplicados detectados: { apt29, dukes }

O nó `dukes` foi introduzido propositadamente no dataset para simular uma segunda fonte de inteligência que reporta o mesmo ator sob o nome *The Dukes*, com `alias` = "Cozy Bear". O algoritmo deteta corretamente que ambos partilham o `alias` e os agrupa como potenciais duplicados — situação frequente em CTI real, onde diferentes feeds usam nomenclaturas distintas para o mesmo ator.

4 Consultas Analíticas

4.1 Atores que usam determinado malware

Malware	Atores
sunburst	apt29 (APT29/Cozy Bear), dukes (The Dukes)
wannacry	lazarus (Lazarus Group)

A consulta para `sunburst` devolve dois atores porque o dataset contém intencionalmente o nó duplicado `dukes`. Após resolução de entidades, conclui-se que ambos representam o mesmo ator.

4.2 Indicadores que apontam para determinada ameaça

Ameaça	Indicadores (IoCs)
sunburst	ioc-domain-avsvmcloud (avsvmcloud[.]com) ioc-hash-sunburst (SHA-256 da DLL maliciosa)

Estes dois IoCs são suficientes para detetar a presença do SUNBURST numa rede: o hash identifica o binário e o domínio identifica o servidor C2.

5 Cenário de Investigação

O sistema foi demonstrado com o seguinte cenário narrativo, típico de uma investigação CTI:

1. Um analista deteta nos logs o hash `32519b85...` (indicador `ioc-hash-sunburst`).
2. O sistema resolve: o indicador *indica* o malware `SUNBURST`.
3. Consulta analítica: que atores usam o `SUNBURST`? → `apt29` e `dukes`.
4. Resolução de entidades: `apt29` e `dukes` são o mesmo ator (APT29/Cozy Bear/The Dukes).
5. Caminho mais curto que justifica a atribuição:

`ioc-hash-sunburst` → `sunburst` → `apt29`

6. Conclusão: o incidente é atribuível ao APT29, grupo de espionagem russo que comprometeu agências do governo norte-americano através da supply chain da SolarWinds.

Este cenário demonstra como o grafo transforma dados dispersos (um hash isolado) em conhecimento estruturado e acionável (atribuição a um ator com justificação pelo caminho de relações).

6 Testes

A suite de testes automáticos (`./cti_graph -testes`) cobre:

- Inserção e validação de nós e relações
- Vizinhos e filtragem por tipo de entidade
- Integridade do dataset sintético
- Persistência SQLite (round-trip guardar/carregar)
- BFS, DFS, caminho mais curto, PageRank
- Resolução de entidades e detecção de duplicados
- Consultas analíticas

Todos os testes passaram com sucesso na execução documentada neste relatório.

7 Conclusões

O Nível 3 transforma o sistema de um repositório passivo num motor de análise. Os principais resultados são:

- O **SUNBURST** é a entidade mais central do grafo (PageRank = 0.1147), refletindo o papel pivô que desempenhou no ataque SolarWinds como ponto de convergência entre o ator, a campanha e os indicadores.
- O **caminho mais curto** permite justificar atribuições de forma auditável: em apenas dois saltos é possível ligar um IoC ao ator responsável.
- A **resolução de entidades** é essencial em CTI real, onde o mesmo ator é frequentemente reportado sob nomes diferentes por fontes distintas. O mecanismo implementado deteta corretamente o alias *Cozy Bear* como denominador comum entre **apt29** e **dukes**.
- A combinação de BFS/DFS com consultas analíticas permite responder às perguntas fundamentais de threat intelligence: *quem usa o quê*, *o que aponta para quê*, e *qual o caminho entre dois pontos do grafo*.